Shenzhen Hiwonder Technology Co,Ltd

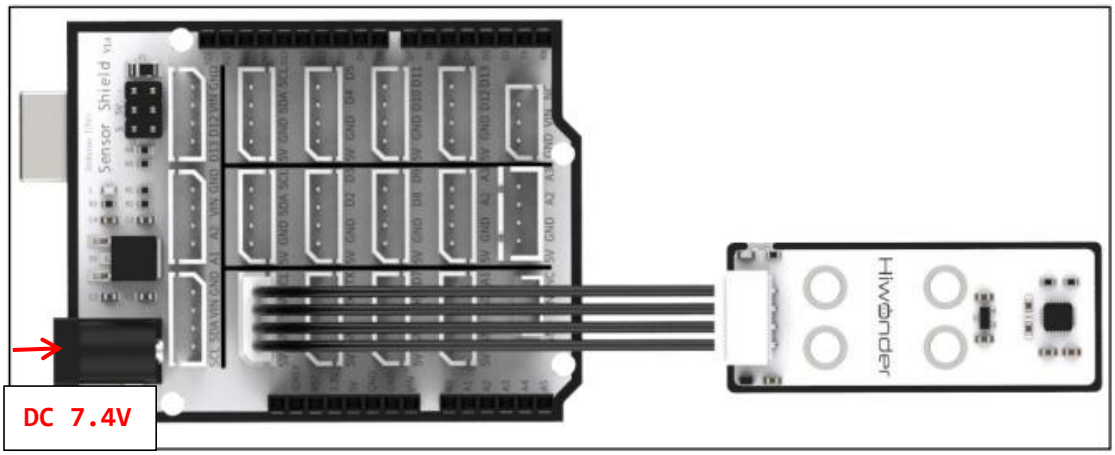
Arduino Development Tutorial

V1.0

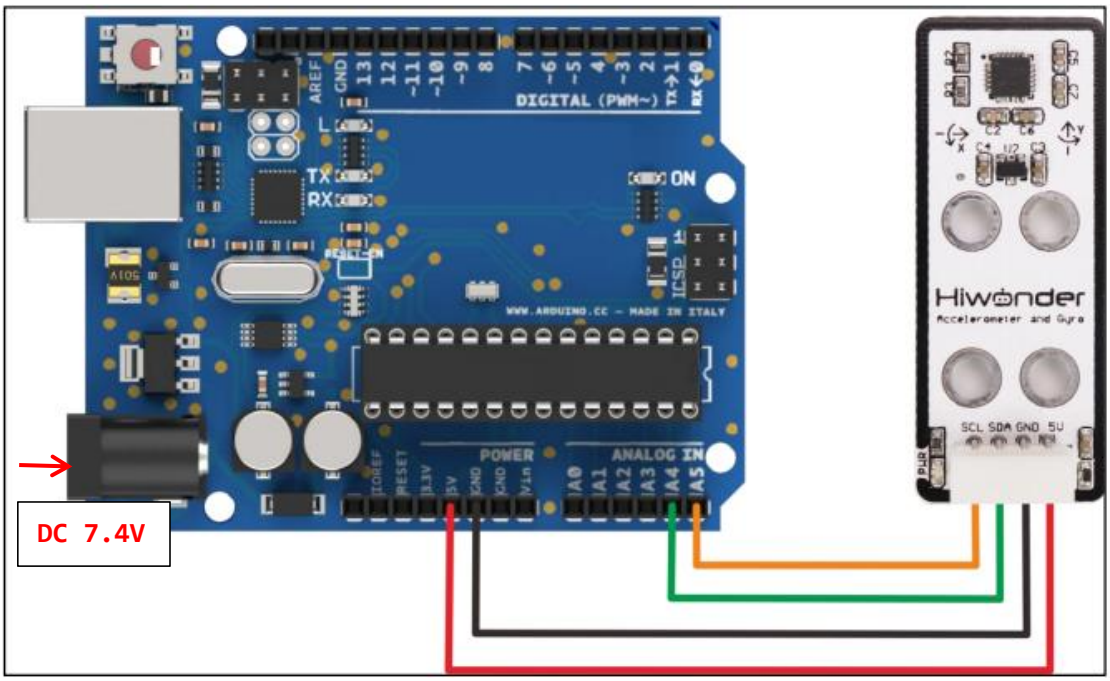
1. Getting Ready

1.1 Wiring Instruction

This section illustrates connecting to SDA and SCL on the Arduino expansion board using a 4-pin cable. Please refer to the diagram below:



If you don't have Arduino expansion board, please directly connect the sensor to the Arduino demo board with DuPont wire, as shown below:



Note:

① If you are using the lithium battery provided by our company, please connect its red wire to the positive terminal (+) of the DC port, and its black wire to the negative terminal (-).

② If the lithium battery wires are not connected, do not directly connect them to other wires to avoid a short circuit between the positive and negative poles, which may cause damage to the device.

③ Before powering on, please ensure no metal objects come into contact with the master. This action prevents the pins at the bottom of the master from a short circuit, which may cause damage to the master.

1.2 Environment Configuration

To install Arduino IDE on your PC, you can access the software package in “2. Software Tools&Program Collections->01 Software Installation Packages-> Arduino Installation Package”. For detailed instructions on how to use Arduino IDE, please refer to the corresponding directory.

2. Case Test

This case uses a terminal window to display the detected values of the acceleration sensor.

2.1 Program Download

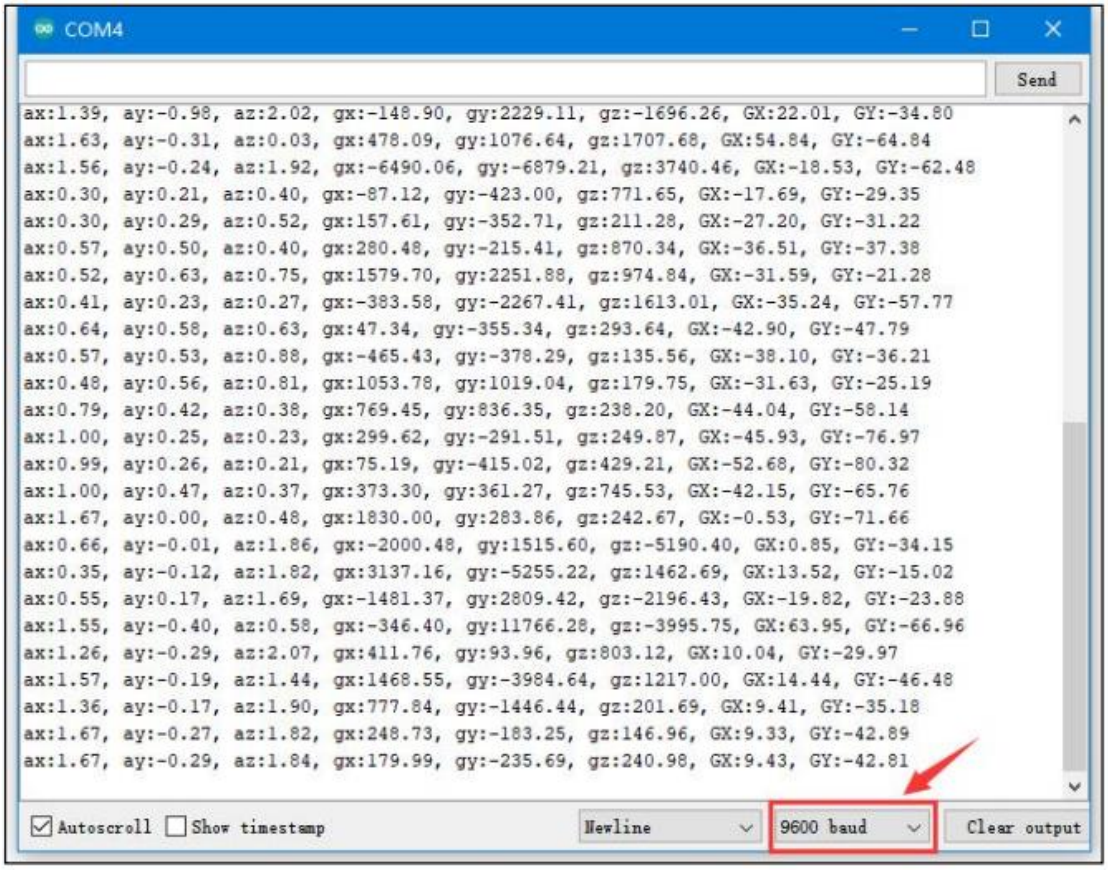
1) Connect the Arduino UNO demo board with the expansion board to the computer using a USB cable. Open Arduino IDE and click “File->New” to import the program from this tutorial directory.

2) Select the suitable demo board and port, then compile and upload the program. The following port is for reference only, please rely on the actual connection.

3) After the code is uploaded successfully, click to open the serial monitor and set the baud rate to 9600. Then, observe the phenomenon.

2.2 Program Outcome

When the sensor is moved, the serial monitor will constantly display the changing data. ax, ay and az respectively represent gravitational acceleration multiples of X-axis, Y-axis and Z-axis of the sensor. gx, gy and gz respectively represent angular velocity of X-axis, Y-axis and Z-axis. GX and GY represent the inclination angle of the X-axis and Y-axis.



2.3 Brief Program Analysis

- **Add Library File**

```
#include <MPU6050.h>   //Add library file of MPU6050

#include <I2Cdev.h>   //Add library file of I2C communication
```

- **Define Variable**

```
MPU6050 accelgyro;

int16_t ax, ay, az;   //Obtain acceleration value of 3-axis of the
sensor

int16_t gx, gy, gz;   //Obtain inclination angle value of 3-axis of the
sensor

float ax0, ay0, az0;  //Store inclination angle value after filtering

float gx0, gy0, gz0;  //Store inclination angle value after filtering

float ax1, ay1, az1;  //Store corrected acceleration value
```

```
float gx1, gy1, gz1;  //Store corrected angular velocity value
float dx;
float dz;
/*Store deviation value between inclination angle and acceleration*/
int ax_offset, ay_offset, az_offset, gx_offset, gy_offset, gz_offset;
float radianX; //Record inclination angle of X-axis
float radianY; //Record  inclination angle of Y-axis
float radianZ; //Record inclination angle of Z-axis
float radianX_last;   //Final obtained X-axis inclination angle
float radianY_last;   //Final obtained Y-axis inclination angle
```

Define the variables to be used in calculating.

- **Update Inclination Angle Data of the Sensor**

```
void update_mpu6050()
{
  static uint32_t timer_u;
  if (timer_u < millis())
  {
    // put your main code here, to run repeatedly:
    timer_u = millis() + 20;
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    ax0 = ((float)(ax)) * 0.3 + ax0 * 0.7;  //Filter read value
    ay0 = ((float)(ay)) * 0.3 + ay0 * 0.7;
    az0 = ((float)(az)) * 0.3 + az0 * 0.7;
    ax1 = (ax0 - ax_offset) /  8192.0;  //
Calibrate and change to multiples of gravitational acceleration
    ay1 = (ay0 - ay_offset) /  8192.0;
    az1 = (az0 - az_offset) /  8192.0;
```

```
        gx0 = ((float)(gx)) * 0.3 + gx0 * 0.7; //Filter read value of angular
velocity
        gy0 = ((float)(gy)) * 0.3 + gy0 * 0.7;
        gz0 = ((float)(gz)) * 0.3 + gz0 * 0.7;
        gx1 = (gx0 - gx_offset);  //calibrate angular velocity

        gy1 = (gy0 - gy_offset);
        gz1 = (gz0 - gz_offset);

        //Complementary calculation for x-axis inclination angle
        radianX = atan2(ay1, az1);
        radianX = radianX * 180.0 / 3.1415926;
        float radian_temp = (float)(gx1) / 16.4 * 0.02;
        radianX_last = 0.8 * (radianX_last + radian_temp) + (-radianX) * 0.2;

        //Complementary calculation for y-axis inclination angle
        radianY = atan2(ax1, az1);
        radianY = radianY * 180.0 / 3.1415926;
        radian_temp = (float)(gy1) / 16.4 * 0.01;
        radianY_last = 0.8 * (radianY_last + radian_temp) + (-radianY) * 0.2;
      }
    }
```

- **Print Data**

```
    void print_data()
    {
      static uint32_t timer_p;
      static uint32_t timer_printlog;
      if ( timer_p < millis())
```

```
    {
      Serial.print("ax:"); Serial.print(ax1);
      Serial.print(", ay:"); Serial.print(ay1);
      Serial.print(", az:"); Serial.print(az1);
      Serial.print(", gx:"); Serial.print(gx1);
      Serial.print(", gy:"); Serial.print(gy1);
      Serial.print(", gz:"); Serial.print(gz1);
      Serial.print(", GX:"); Serial.print(radianX_last);
      Serial.print(", GY:"); Serial.print(radianY_last);
      timer_p = millis() + 500;
    }
  }
```

- **Serial Initialization**

```
  void setup()
  {
    Serial.begin(9600);

    //MPU6050 Configuration
    Wire.begin();
    accelgyro.initialize();
    accelgyro.setFullScaleGyroRange(3); //Set angular velocity range
    accelgyro.setFullScaleAccelRange(1); //Set acceleration range
    delay(200);
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);  //Obtain current
3-axis data for calibration
    ax_offset = ax;  //Calibration data of X-axis acceleration
    ay_offset = ay;  //Calibration data of Y-axis acceleration
    az_offset = az - 8192;  //Calibration data of Z-axis acceleration
```

```
  gx_offset = gx; //Calibration data of X-axis angular velocity
  gy_offset = gy; //Calibration data of Y-axis angular velocity
  gz_offset = gz; //Calibration data of Z-axis angular velocity
  delay(1500);
}
```

- **Loop Execution**

```
void loop()
{
  update_mpu6050();
  print_data();
  delay(500);
}
```